

项目详解：BigTree Discord RAG Bot

本文档说明项目整体设计、架构与模块职责。完整功能清单见 `FEATURE_LIST.md`；快速上手见 `SETUP_AND_TEST.md`。

目录

- 一、设计思路
 - 要解决什么问题？
 - 为什么选 RAG，而不是微调？
 - 四条核心设计原则
 - 回答边界（故意不做事）
 - 核心问答流程
 - 知识如何变「像你」
 - 二、技术架构
 - 分层总览
 - 启动顺序（`bot/main.py`）
 - 启动时加载的主要 Cog
 - 问答主链路（详细）
 - RAG 与置信度如何配合
 - 外部依赖与本地状态
 - 后台任务一览
 - 管理端与客户端（可选表面）
 - 三、功能全景
 - 四、核心模块说明
 - 配置与入口
 - 数据导入层
 - 消息处理与 RAG
 - 审核与学习闭环
 - 内容与资讯自动化
 - 获客与推广
 - 安全与运维
 - 管理端与客户端
 - 五、关键技术概念
 - 六、费用结构
 - 七、安全与可靠性设计
 - 八、项目文件结构
 - 九、日常使用命令速查
 - 十、Slash Commands 速查
 - 十一、环境安装与初始化
-

一、设计思路

要解决什么问题？

你有一个大型 Discord 股票/信号社区，成员全天候提问。频道主无法 24 小时在线，但又希望：

1. 自动回复尽量像自己的语气与观点（不是泛泛的 AI 理财话术）
2. 不确定的回答先人工审核，避免乱答、乱给进场点
3. 知识库持续增长（频道主发言、审核通过内容、YouTube 字幕等）
4. 运营自动化：日/周总结、金十快讯、视频摘要、促销排程、新人转化

同时要兼顾：限速防刷、垃圾/诈骗过滤、主题跑偏控制、以及 OpenAI 费用与延迟可控。

为什么选 RAG，而不是微调？

方案	优点	对本项目的问题
微调 (Fine-tune)	语气可固化进权重	观点天天变；每次新内容要重训；成本高、迭代慢
RAG (本项目)	新发言入库即可用；可追溯「答了哪段历史」；易关停/灰度	依赖检索质量与 prompt 约束

结论：用 ChromaDB 存你的历史内容 + GPT 按风格指南作答。风格来自 data/style_profile.txt（可由 ingestion.analyze_style 生成），不是改模型权重。早期规划里也明确排除了「先上微调」。

四条核心设计原则

1. 检索增强，而不是瞎编

先 Embedding 检索相关历史，再生成。检索为空或距离过远时，宁可交给频道主，也不硬答。

2. 置信度三重门，默认偏保守

LLM 自评 CONFIDENCE: 1 - 10（默认阈值 7）+ 检索质量（无上下文 / best_distance 过大）+ 领域硬规则（如「现在能不能进 / 给点位」类信号问题一律审核，因为 Bot 看不到实时盘面）。

3. 人机闭环持续变准

- Owner 在目标频道发言 / 语音 → 自动入库
- 审核 Approve / Edit → Q&A 入库
- Reject → 写入负样本，注入后续 system prompt
- 成员 [up][down]、Edit → 反馈与待补知识缺口队列

4. 能力可开关、可灰度

多数子系统默认关闭（金十、YouTube、日/周总结、AutoMod、Admin 等），用 env 打开；新能力还有 Feature Flag + 按频道 Canary，避免一次全服翻车。

回答边界（故意不做事）

- 不假装能看盘：信号/进场时机类问题强制进 Owner 审核。
- 不打断闲聊：默认 RESPOND_MODE=questions，只回答问题、@Bot、回复 Bot、图片/语音等。
- 不替代促销话术乱插：购买意向走独立 CTA 路径，与 RAG 分流。
- 价格点位会脱敏：生成后对危险价位表述做 redaction；可选 Safety Guardrails（强制审核或加免责声明）。

核心问答流程

成员提问：“AAPL怎么看？”



① on_message 预检

- 目标频道？排除频道？机器人？寒暄/刷屏/外链？
- Owner 发言 → 只学习入库，不走问答
- 购买意向 → 产品 CTA（跳过 RAG）
- 限速（VIP 角色可豁免）→ 进入异步队列



② 队列单线程处理（避免并发打爆 API / 乱序回复）

- Thread / 近期对话记忆；过长则 Session 摘要压缩
- 语音 → Whisper；图片 → Vision（可带少量检索上下文）



③ RAG

- Embedding → ChromaDB top-k → 距离过滤
- System: 风格指南 + 负样本提示
- User: 检索片段 + 对话历史 + 问题
- 输出回答 + CONFIDENCE



④ confidence.route_answer

- 达标 → 频道自动回复（可附带低频 CTA）
- 未达标 / 信号题 / 无上下文 → Owner 私信审核

↓

⑤ 闭环

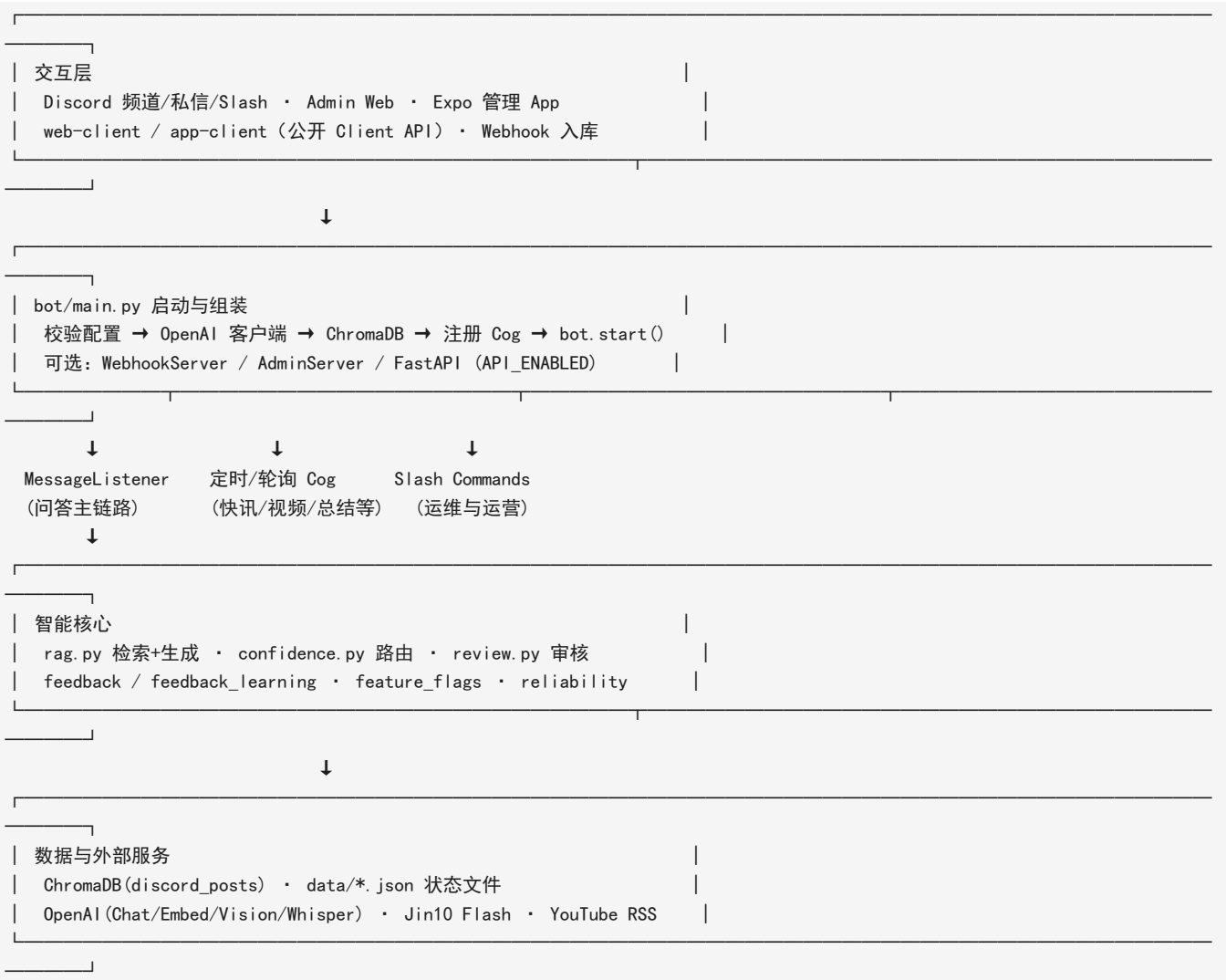
- Approve/Edit → 入库; Reject → 负样本
- [up][down] → feedback; 缺口 → learning_queue

知识如何变「像你」

历史 Discord 导出 / 增量 — ingestion. ingest — ChromaDB
 Owner 日常发言 / 语音 — listener 实时学习 — ChromaDB
 审核通过的 Q&A — review 入库 — ChromaDB
 YouTube 字幕（可选） — ingest_youtube — ChromaDB
 风格分析 — analyze_style — style_profile.txt
 拒绝的坏回答 — negative_samples — 下次 prompt 约束

二、技术架构

分层总览



启动顺序 (bot/main.py)

1. `_validate_config()`: 至少需要 DISCORD_BOT_TOKEN、OPENAI_API_KEY

2. 创建 `openai.AsyncOpenAI`（超时与重试策略由 RAG 层再包一层）
3. 注入 OpenAI 到禁止词 / Topic Guard；加载 `ban_words`
4. `chromadb.PersistentClient(CHROMADB_PATH)` → 集合 `discord_posts (cosine)` → `AsyncCollection` 包装
5. 创建 Discord Bot（需 `message_content / members / invites intents`）
6. 注册 `on_ready`（公会级 + 全局 Slash 同步）
7. 按序加载 Cog（见下表）
8. 按开关拉起 Webhook / Admin HTTP / FastAPI
9. `bot.start(...)`；进程信号触发优雅卸载

MessageListener 就绪后还会：启动消息队列 worker、按需离线回填、周期性落盘 `last_seen`。

启动时加载的主要 Cog

顺序	Cog	作用
1	MessageListener	频道监听、RAG 问答、离线回填、学习 Owner 发言
2 - 3	PromotionCommands / BotCommands	Slash：推广 + 运维
4	SchedulerCog	促销 / 教学 / 提醒 / FAQ 推送 / SLA tick
5	HealthCog	健康与状态
6	IngestionSchedulerCog	定时跑 <code>ingestion.ingest + analyze_style</code>
7	DigestCog	频道活动摘要（可选）
8	NewsFeedCog	金十快讯轮询与离线回填
9	YouTubeMonitorCog	新视频 → 排程 / 入库 / 摘要
10	AcquisitionCog	欢迎 drip、邀请追踪
11	PromoMonitorCog	源频道活动 → 自动排程促销
12 - 13	WeeklySummaryCog / DailySummaryCog	周/日重点总结
14	AutoModCog	垃圾/诈骗、禁止词、主题限制

问答主链路（详细）

```

Discord on_message
├── Owner 文本/语音（目标频道）→ 学习入库 → return
├── 更新 last_seen；关键词告警 / 好评采集（旁路）
├── _should_skip（机器人、排除频道、寒暄、刷屏、非问题模式等）
├── 限速（非 VIP）→ asyncio.Queue
└── ↓
单 worker _handle_message
├── 组装 Thread / 频道对话记忆（可 Session 摘要）
├── 语音 Whisper / 图片 Vision
├── 购买意向？→ CTA Embed + 漏斗埋点 + 通知 Owner（不进 RAG）
└── ↓
run_rag_pipeline (rag.py)
├── embed 问题 → 查 Chroma top_k → 丢弃 distance > RAG_MAX_DISTANCE
├── 无命中 → 固定「不确定」话术 + 低置信度
├── 有命中 → GPT 生成（风格 + 负样本 + 历史）→ 解析 CONFIDENCE → 价位脱敏
└── ↓
route_answer (confidence.py)
├── 信号题 / 无上下文 / 距离过差 / 分低于阈值 → forward_to_owner
├── 否则 auto_reply（可再被 Guardrails / Clarification 改写）
└── ↓
auto_reply → 频道回复 + 可选 CTA + 通知 Owner（知情 DM）
或
send_for_review → Owner DM 按钮 + review_queue.json + （可选）WebSocket 推管理端
    
```

RAG 与置信度如何配合

环节	关键配置 / 行为
----	-----------

检索	EMBEDDING_MODEL (默认 text-embedding-3-small)、RAG_TOP_K (默认 8)、RAG_MAX_DISTANCE (默认 0.6)
生成	LLM_MODEL (默认 gpt-4o-mini); Vision 用 VISION_MODEL (默认 gpt-4o)
风格	style_profile.txt, 缓存 TTL 约 5 分钟
自评	回答末尾 CONFIDENCE: N; 解析失败按 3 分处理
路由	默认 CONFIDENCE_THRESHOLD=7; best_distance 0.95 即使高分也转审; context_count==0 转审
澄清	FEATURE_CLARIFICATION_FOLLOWUP: 低置信但若仍自动回, 可改成追问澄清

外部依赖与本地状态

类型	内容
Discord	Gateway、消息、私信、Slash、角色/邀请、附件
OpenAI	Chat、Embedding、Vision、Whisper; 禁止词/主题语义辅助
Jin10	Flash API 轮询 (NEWS_POLL_INTERVAL_SECONDS, 重要快讯优先)
YouTube	Channel RSS; yt-dlp/字幕或 Whisper 入库; GPT 摘要
ChromaDB	CHROMADB_PATH, 集合 discord_posts
data/*.json	游标与运营状态: last_seen、jin10_last_id、review_queue、feedback、negative_samples、promos、funnel、stats 等

后台任务一览

模块	触发方式	做什么
NewsFeedCog	轮询 + 每次 on_ready 回填	金十 → Discord; 发帖成功后才推进 last_id
YouTubeMonitorCog	定时 / 日检	新片 → 教学排程 + 可选入库 + 摘要频道
DailySummaryCog / WeeklySummaryCog	按 ET 钟点	扫 Owner 发言 → GPT → 推送 (防重复 / 可补跑)
SchedulerCog	约每 60s	到期促销/教学/提醒、FAQ 推送、SLA
PromoMonitorCog	源频道 Owner 消息	取消旧自动促销, 排新的重复推送
AcquisitionCog	成员事件 + drip	欢迎序列、邀请归因
IngestionSchedulerCog	间隔任务	子进程增量入库 + 风格重分析
Listener 回填	启动且开关打开	补答离线期间漏掉的问题, 并学习错过的 Owner 发言

管理端与客户端 (可选表面)

表面	入口	用途
Admin Web	ADMIN_ENABLED + ADMIN_PORT	浏览器看状态 / 配置 / KB
FastAPI 管理 API	API_ENABLED (默认常开)	供 app/ Expo 管理端: 审核、KB、配置; WebSocket 推送待审
公开 Client API	CLIENT_API_ENABLED	/api/public/*: 聊天、新闻、总结、搜索等
web-client / app-client	Vite / Expo	终端用户界面 (不经 Discord 登录)

核心问答链路不依赖这些表面: 关掉 API/Admin 后, 纯 Discord Bot 仍可独立运行。

三、功能全景

按能力分组（编号对应 FEATURE_LIST.md 中的条目，便于对照）：

A. 智能问答核心

能力	要点
RAG 问答	ChromaDB + GPT，风格指南驱动
置信度路由	自动回复 / Owner 审核
离线回填	重启后补答目标频道漏掉的问题
自动学习 Owner 消息	频道主发言入库
Thread / 图片 / 语音	Thread 上下文；Vision；Whisper 转写
多语言检测	中/英/日/韩等同语种回复
Clarification / Session 摘要	低置信澄清追问；长对话压缩记忆
Feature Flags + Canary	按频道灰度开关新能力

B. 内容与资讯自动化

能力	要点
金十快讯	轮询 Flash API；重要 Embed；重连回填；成功发帖后才推进 last_id
日总结 / 周总结	扫描频道主发言；GPT 摘要；防重复发送与错过补跑
YouTube Monitor	RSS 新片 → 教学排程 + 可选入库 + 摘要频道；/resend_summary 补发
Digest	每日活动摘要（另通道）

C. 获客与推广

能力	要点
促销 / 试用 / 教学排程	重复模式：不重复 / 每小时 / 天 / 周 / 月
Promo Monitor	源频道发活动 → 自动每日推送（默认 ET 16:00）
欢迎 drip	即时欢迎 → 价值内容 → 产品 CTA → 第 3 天提醒
购买意向转化	询价/订阅等关键词 → 产品 Embed + CTA + 通知 Owner
邀请裂变 /invite	专属邀请链接 + 奖励阈值
自愿通知身份组私信	仅征得同意的角色可 DM；/promo_notify_panel
/funnel	转化漏斗看板

D. 安全与质量

能力	要点
Auto Mod	关键词 / 禁止词（精确+语义）/ 外链 / 邀请 / 刷屏
Topic Guard	指定频道只允许投资相关讨论（GPT 分类）
Safety Guardrails	高风险表述强制审核或加免责声明
SLA 监控	p95 延迟、OpenAI 错误率、审核积压告警
负反馈学习	Reject / 踩 → 负样本与待补 KB 池

E. 管理面

Admin Web、FastAPI API、Expo Admin App、Web/移动客户端、Webhook 入库、KB
快照、导出对话、满意度与排行榜等。完整列表见 FEATURE_LIST.md。

四、核心模块说明

配置与入口

bot/config.py — 从 .env 读取全部开关与阈值（RAG、频道、新闻、YouTube、总结、获客、AutoMod、Feature Flags、SLA 等）。

bot/main.py — 校验密钥 → 初始化 OpenAI / ChromaDB → 注册 Cog → Guild 级 slash sync（重启后命令尽快可见）→ 登录 Discord。

常用核心配置示例：

配置项	说明
TARGET_CHANNEL_IDS	RAG 监听频道
CONFIDENCE_THRESHOLD	自信度阈值（默认 7）
RESPOND_MODE	auto / review / questions / mention_only
NEWS_FEED_ENABLED / NEWS_CHANNEL_IDS	金十推送
DAILY_SUMMARY_* / WEEKLY_SUMMARY_*	日/周总结时间与频道
YOUTUBE_MONITOR_ENABLED	新视频监控
AUTO_MOD_ENABLED	自动审核
WELCOME_FLOW_ENABLED	新人欢迎与 drip

数据导入层

ingestion/preprocess.py — 预处理

Discord 导出 JSON → 问答对提取 → 连续消息合并 → 文本清理 → 按 token 切块。

ingestion/ingest.py — 向量化

批量 Embedding → 写入 ChromaDB；支持增量（不重复写入）。

ingestion/analyze_style.py — 风格分析

统计长度、常用短语、表情等 → data/style_profile.txt，供 RAG prompt 使用。

ingestion/ingest_youtube.py — YouTube 导入

字幕优先；无字幕则 yt-dlp + ffmpeg + Whisper。二进制可放在 .venv/Scripts/，代码按完整路径解析，不依赖系统 PATH。另支持 PDF 导入（ingest_pdf.py）。

消息处理与 RAG

bot/listener.py — 消息总控

1. 过滤：机器人、排除频道、空内容等
2. AutoMod / Topic Guard：垃圾与跑题先处理
3. 意图：购买意向 → 获客 CTA（可不走 RAG）
4. 限速：用户冷却 + 全局上限；VIP 角色可豁免
5. 多模态：语音条 Whisper；图片 Vision
6. 队列：asyncio.Queue 串行处理，避免打爆 API
7. Owner 发言学习、离线回填、Thread 上下文、反馈反应收集

bot/rag.py — 检索与生成

检索：embedding → Top-K → 距离过滤 → 去重。

生成：风格指南 + 上下文 + （可选）负样本 / Session 摘要 → LLM → 解析 CONFIDENCE。

含嵌入缓存、重试与可靠性埋点。

bot/confidence.py — 路由

典型三重条件（均可配置）：有上下文、最佳距离够近、自信度 $\geq$ 阈值。全满足才自动发；否则进审核。可与 Clarification、Guardrails 叠加。

审核与学习闭环

bot/review.py + bot/review_queue.py

低置信草稿 DM Owner: Approve / Edit / Reject。Approve 可入库; Reject / Edit 进入负样本或学习队列。队列有过期清理, 避免 SLA「积压」误报。

bot/feedback.py / bot/feedback_learning.py

[up][down] 记录满意度; 定期汇总知识缺口 Top N, 推动补库。

内容与资讯自动化

bot/news_feed.py — 金十快讯

- 默认约 30 秒轮询; NEWS_IMPORTANT_ONLY 可只推重要快讯
- 去重、广告过滤; 重要消息用 Embed
- 每次 on_ready (含重连) 回填离线窗口 (默认 24h, 最多 50 条重要)
- 发帖成功后才推进 last_id, 避免漏发

bot/daily_summary.py / bot/weekly_summary.py

- 扫描配置频道内频道主消息与回复, GPT 生成 Embed, 可 @everyone
- 日总结: 防同一天重复发送 (定时器提前唤醒也不会双发)
- 周总结: 读取失败会重试与补跑, 避免「无消息」误判

bot/youtube_monitor.py

- 定时查频道 RSS; 新片可排程教学帖、入库、摘要到 YOUTUBE_SUMMARY_CHANNELS
- 转录失败仍可发新视频通知; /resend_summary 可补摘要 (自动查库 / Whisper 入库 / 标题兜底)
- 摘要可附获客 CTA 按钮

bot/views_summary.py + /views

全服扫描频道主近期发言, 生成简短观点总结发到当前频道 (不 @everyone)。

获客与推广

模块	作用
promo_config.py	CTA / 产品 Embed / 欢迎文案 helper
scheduler.py	促销、教学、提醒到点发送
promo_monitor.py	源频道活动自动转每日排程
testimonials.py	好评检测与审核转发
welcome_flow.py	欢迎 DM + 多日 drip (持久化 welcome_drip.json)
acquisition.py / acquisition_cog.py	意向转化、邀请追踪、漏斗
role_dm.py	自愿通知身份组促销私信 (合规群发)

详细运营玩法见 GROWTH_PLAYBOOK.md、PROMOTION_GUIDE.md。

安全与运维

模块	作用
auto_mod.py / ban_words.py	多层垃圾过滤; 禁止词精确 + 语义相似
topic_guard.py	指定频道主题限制
guardrails.py	高风险表述处理
reliability.py	SLA 指标与 Owner 告警

health.py	健康检查 Cog
feature_flags.py	功能开关与 canary 频道
keyword_alert.py	关键词 DM 告警 Owner
kb_versioning.py	知识库快照

管理端与客户端

路径	说明
bot/admin.py	浏览器 Admin 面板
bot/api/	管理 API + 公开 Client API + WebSocket
app/	Expo Admin 移动端
app-client/ / web-client/	终端用户客户端
bot/webhook.py	HTTP 入库

五、关键技术概念

Embedding（向量嵌入）

文字 → 高维向量；语义相近则向量相近。本项目默认使用 OpenAI embedding 模型。

ChromaDB

本地持久化向量库（chromadb_store/），按相似度检索历史帖子 / 视频转录等。

Cosine Distance

距离	含义
0.0	几乎相同
~0.3	很相关
~0.6	边缘相关
≥0.8	通常过滤掉

RAG

先检索可信上下文，再生成回答，减少「纯模型瞎编」。

Feature Flag / Canary

新能力可先只在部分频道开启，验证稳定后再全量。

六、费用结构

项目	量级	说明
历史入库（一次性 / 增量）	较低	Embedding 按 token
每次文字问答	约 \$0.001 级	检索 + GPT-4o-mini
图片 Vision / 语音 Whisper	更高	按次 / 按时长
金十轮询	近乎 \$0	外部 HTTP，无 OpenAI
日/周总结、视频摘要	中等	依赖扫描消息量与模型
AutoMod 语义禁止词 / Topic Guard	中等	嵌入或 GPT 分类

实际费用随提问量、多模态比例、总结频道数变化。限速与 VIP 豁免用于控制成本。

七、安全与可靠性设计

机制	说明
置信度三重检查	降低胡答进公频概率
Owner 审核队列	不确定内容先私信
限速 + VIP	防刷与控费
AutoMod / Topic Guard	垃圾、诈骗、跑题
Guardrails	all-in / 保证收益等强制审核或免责
密钥仅 .env	不提交 Git
News last_id 成功后推进	避免离线漏发
日总结去重 / 周总结重试补跑	避免漏发或双发
SLA 告警	延迟、错误率、积压通知 Owner
审核按钮防重复 / 超时失效	防双发与僵尸任务

八、项目文件结构

```
treeProjectDiscordBot/
├── .env / .env.example
├── requirements.txt
├── Dockerfile / docker-compose.yml
├── keep_awake.py
├──
├── bot/                                ← 运行时
│   ├── main.py                        ← 入口
│   ├── config.py                     ← 配置
│   ├── listener.py                   ← 消息总控
│   ├── rag.py / confidence.py / review*.py
│   ├── news_feed.py                  ← 金十
│   ├── daily_summary.py / weekly_summary.py / views_summary.py
│   ├── youtube_monitor.py
│   ├── auto_mod.py / ban_words.py / topic_guard.py / guardrails.py
│   ├── acquisition*.py / welcome_flow.py / role_dm.py
│   ├── scheduler.py / promo_*.py / testimonials.py / commands.py
│   ├── reliability.py / feature_flags.py / feedback*.py
│   ├── api/ / admin.py / webhook.py
│   └── ...
├── ingestion/                          ← 离线导入
│   ├── preprocess.py / ingest.py / analyze_style.py
│   └── ingest_youtube.py / ingest_pdf.py
├──
├── scripts/                           ← 运维脚本（补发摘要、回测等）
├── tests/                             ← pytest
├── data/                              ← 运行时 JSON / 风格 / 状态
├── docs/                              ← 本指南与专题文档
├── app/ / app-client/ / web-client/
├── chromadb_store/
└── logs/
```

专题文档：FEATURE_LIST.md、PROMOTION_GUIDE.md、GROWTH_PLAYBOOK.md、USER_GUIDE.md、CLIENT_USER_GUIDE.md、API_DESIGN.md 等。

九、日常使用命令速查

```
# 激活虚拟环境
. venv\Scripts\Activate.ps1

# 启动 Bot
python -m bot.main

# 导入 Discord 数据（增量）
python -m ingestion.ingest

# 导入 YouTube
python -m ingestion.ingest_youtube --urls "https://youtu.be/VIDEO_ID"

# 风格分析
python -m ingestion.analyze_style

# 测试
python -m pytest tests/ -v

# 知识库文档数（集合名以实际为准，常见 bigtree_knowledge）
python -c "import chromadb; c=chromadb.PersistentClient('./chromadb_store'); print([x.name for x in c.list_collections()])"

# 防休眠
python keep_awake.py
```

补发 YouTube 摘要（脚本备用）：

```
python scripts/resend_youtube_summary.py
python scripts/resend_youtube_summary.py --video-id VIDEO_ID
```

十、Slash Commands 速查

公开

命令	说明
/ask	RAG 提问
/signal	产品介绍
/invite	专属邀请链接
/testimonials	用户好评
/faq	FAQ
/promo_notify	领取/取消活动私信通知

Owner — 内容与总结

命令	说明
/daily_summary / /weekly_summary	立即推送日/周总结
/views	全服频道主观点简短总结
/resend_summary	补发 YouTube GPT 摘要
/pin_summary	总结并钉选近期讨论
/generate_faq	生成 FAQ

Owner — 推广与获客

命令	说明
/post_promo / /schedule_promo	发/排程促销（可选 dm_role）

/schedule_trial / /schedule_lesson	试用回顾 / 教学
/list_promos / /cancel_promo 等	管理排程
/promo_notify_panel / /dm_role	订阅面板 / 自愿角色私信
/funnel	转化漏斗

Owner — 运维与质量

命令	说明
/status / /stats / /kb_report / /search_kb	状态与知识库
/satisfaction / /leaderboard / /ab_results	反馈与实验
/add_ban_word / /list_ban_words 等	禁止词
/add_alert / /list_alerts 等	关键词告警
/schedule_reminder 等	自定义提醒
/export_conversations / /kb_snapshots	导出与快照

排程重复模式：不重复 / 每小时 / 每天 / 每周 / 每月。

命令以 Bot 当前注册为准；完整说明见 FEATURE_LIST.md。

十一、环境安装与初始化

快速上手也可直接看 SETUP_AND_TEST.md。

前置条件

组件	要求	验证命令
Python	3.11+	python --version
Node.js	18+（前端需要）	node --version
FFmpeg / deno	YouTube Whisper 路径建议	可放在 .venv\Scripts\
Discord Bot Token	开发者门户	—
OpenAI API Key	OpenAI	—
Git	版本控制	git --version

安装步骤

```
# 1. 系统工具（示例）
winget install --id Python.Python.3.11 -e
winget install --id Gyan.FFmpeg -e

# 2. 克隆并进入项目
git clone <your-repo-url>
cd treeProjectDiscordBot

# 3. 虚拟环境
python -m venv .venv
.venv\Scripts\Activate.ps1

# 4. 依赖
pip install -r requirements.txt

# 5. 配置
Copy-Item .env.example .env
# 必填: DISCORD_BOT_TOKEN, OPENAI_API_KEY, OWNER_USER_ID, TARGET_CHANNEL_IDS

# 6. 入库 +（可选）风格
python -m ingestion.ingest
```

```
python -m ingestion.analyze_style
```

7. 启动

```
python -m bot.main
```

换电脑后重建虚拟环境（Windows）

不要复制旧 .venv。用 py launcher 重建：

```
Remove-Item -Recurse -Force .venv
py -m venv .venv
.venv\Scripts\Activate.ps1
python --version
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m bot.main
```

若 Activate.ps1 被拦截：

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

验证安装

```
python -m pytest tests/ -v
python -c "import chromadb; c=chromadb.PersistentClient('./chromadb_store'); print([x.name for x in c.list_collections()])"
```

正常启动日志大致包含：OpenAI 初始化、ChromaDB 文档数、Bot ready、消息队列 worker 启动。

前端（可选）

```
cd app; npm install; npx expo start
cd app-client; npm install; npx expo start
cd web-client; npm install; npm run dev
```

详见 USER_GUIDE.md、CLIENT_USER_GUIDE.md。